

Chapter 2. User-Item Ratings and Framing the Problem

If you were asked to curate the selection for a cheese plate at a local café, you might start with your favorites. You might also spend a bit of time asking for your friends' favorites. Before you order a large stock in these cheeses for the café, you would probably want to run a small experiment—maybe asking a group of friends to taste your selections and tell you their preferences.

In addition to receiving your friends' feedback, you'd also learn about your friends and the cheeses. You'd learn which kinds of cheeses your friends like and which friends have similar tastes. You can also learn which cheeses are the most popular and which cheeses are liked by the same people.

This data would start to give you hints about your first cheese recommender. In this chapter, we'll talk about how to turn this idea into the right stuff for a recommendation system. By way of this example, we'll discuss one of the underlying notions of a recommender: how to predict a user's affinity for things they've never seen.

The User-Item Matrix

It's extremely common to hear those who work on recommendation systems talk about matrices, and in particular the user-item matrix. While linear algebra is deep, both mathematically and as it applies to RecSys, we will begin with simple relationships.

Before we get to the matrix forms, let's write down some binary relationships between a set of users and a set of items. For the sake of this exam-

ple, think of a group of five friends (mysteriously named *A*, *B*, *C*, *D*, *E*) and a blind cheese tasting where of four cheeses (*gouda*, *chèvre*, *emmentaler*, *brie*). The friends are asked to rate the cheeses, 1–4:

- 1. *A* starts, “OK, I really enjoy *gouda*, so give that a 5; *chèvre* and *emmentaler* are yummy too, 4; and *brie* is awful, 1.”
- 2. *B* replies, “What?! *brie* is my favorite! 4.5! *chèvre* and *emmentaler* are fine, 3; and *gouda* is just OK, 2.”
- 3. *C* gives ratings of 3, 2, 3, and 4, respectively.
- 4. *D* gives 4, 4, 5, but we run out of *brie* before *D* can try it.
- 5. *E* starts to not feel well, and tries only *gouda*, giving it a 3.

The first thing you may notice is that such expository writing is a bit tedious to read and parse. Let’s summarize these results in a convenient table ([Table 2-1](#)):

Table 2-1. Cheeses and ratings

Cheese taster	Gouda	Chèvre	Emmentaler	Brie
A	5	4	4	1
B	2	3	3	4.5
C	3	2	3	4
D	4	4	5	-
E	3	-	-	-

Your first instinct may be to write this in a form more appropriate for computers. You might create a collection of lists:

A : [5, 4, 4, 1]
B : [2, 3, 3, 4.5]
C : [3, 2, 3, 4]
D : [4, 4, 5, –]
E : [3, –, –, –]

This may work in some scenarios, but you might want to more clearly indicate the positional meaning in each list. You could simply visualize this data with a heatmap ([Figure 2-1](#)):

```
import seaborn as sns

_ = np.nan
scores = np.array([[5,4,4,1],
                  [2,3,3,4.5],
                  [3,2,3,4],
                  [4,4,5,_],
                  [3,_,_,_]])
sns.heatmap(
    scores,
    annot=True,
    fmt=".1f",
    xticklabels=['Gouda', 'Chevre', 'Emmentaler', 'Brie',],
    yticklabels=['A', 'B', 'C', 'D', 'E',]
)
```

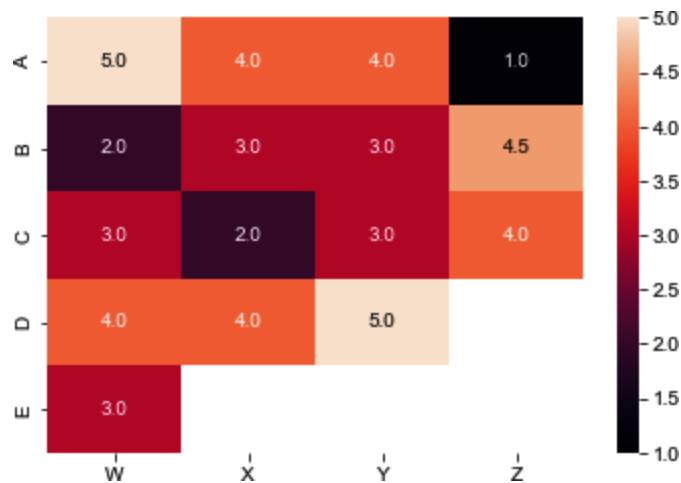


Figure 2-1. Cheese ratings matrix

As we observe datasets with huge numbers of users or items, and with more and more sparsity, we will need to employ a data structure more well suited to representing only the necessary data. A variety of so-called *dense representations* exists, but for now we will use the simplest form: tuples of `user_id`, `item_id`, and `rating`. In practice, the structure is often a dictionary with indices provided by the IDs.

Two types of structures for these kinds of data are dense and sparse representations. Loosely, a *sparse representation* is one such that a datum exists for each nontrivial observation. A *dense representation* always contains a datum for each possibility even when trivial (`null` or zero).

Let's see what this data looks like as a dictionary:

```
'indices': [  
    (0,0),(0,1),(0,2),(0,3),  
    (1,0),(1,1),(1,2),(1,3),  
    (2,0),(2,1),(2,2),(2,3),  
    (3,0),(3,1),(3,2),  
    (4,0)  
]  
'values': [  
    5,4,4,1,  
    2,3,3,4.5,  
    3,2,3,4,  
    4,4,5,  
    3  
]
```

A few natural questions emerge:

1. What's the most popular cheese? From the observations so far, it's looking like *emmentaler* is potentially the favorite, but *E* didn't try *emmentaler*.
2. Would *D* like *brie*? It seems to be a contentious cheese.
3. If you were asked to buy only two cheeses, which should you buy to best satisfy everyone?

This example and associated questions are intentionally simple, but the point is clear that this matrix representation is at least convenient for capturing these ratings.

What may not be obvious is that beyond the convenience of this data visualization is the mathematical utility of this representation. Question 2

suggests an inherent RecSys problem: “predict how much a user will like an item they haven’t seen.” This question may also be recognizable as a problem from a linear algebra class: “How can we fill in unknown elements of a matrix from the ones we know?” This is called *matrix completion*.

The back-and-forth between creating user experiences that capture their needs and the mathematical formulations to model this data and needs is at the heart of recommendation systems.

User-User Versus Item-Item Collaborative Filtering

Before we dive into the linear algebra, let’s consider the purely data science perspective called *collaborative filtering* (CF), a term originally used by David Goldberg et al. in their 1992 paper [“Using Collaborative Filtering to Weave an Information Tapestry”](#).

The underlying idea of CF is that those with similar tastes help others to know what they like without having to try it themselves. The *collaboration* terminology was originally intended to mean among similar-taste users, and *filtering* was originally intended to mean filtering out choices people will not like.

You can think of this CF strategy in two ways:

- Two users with similar tastes will continue to have similar tastes.
- Two items with similar user fans will continue to be popular with other users who are similar to those fans.

These may sound identical, but they appear differently in the mathematical interpretations. At a high level, the difference is in deciding which kind of similarity your recommender should prioritize: user similarity or item similarity.

If you prioritize *user similarity*, then to provide a recommendation for a user *A*, you find a similar user *B* and then choose a recommendation from *B*'s list of liked content that *A* hasn't seen yet.

If you prioritize *item similarity*, then to provide a recommendation for a user *A*, you find an item that *A* liked, *chèvre*, and then you find an item similar to *chèvre* that *A* hasn't seen, *emmentaler*, and recommend it for *A*.

Later we will dive deeper into similarity, but let's quickly link these ideas to our preceding discussion. *Similar users* are rows of the user-item matrix that are similar as vectors; *similar items* are columns of the user-item matrix that are similar as vectors.

VECTOR SIMILARITY

Dot product similarity is more precisely defined in [Chapter 10](#). For now, consider similarity to be computed by normalizing the vectors and then taking their cosine similarity. Given entities of any kind that you've associated to vectors (lists of numbers), *vector similarity* compares which entities are most alike with respect to the characteristics captured by those lists of numbers (called the *latent space*).

The Netflix Challenge

In 2006, Netflix kicked off an online competition called the Netflix Prize. This competition challenged teams to improve on the performance of Netflix CF algorithms on a dataset released as open source by the company. While such a competition is common today via websites like Kaggle or conference, at that time, it was very exciting and novel for those interested in RecSys.

The competition consisted of several intermediate rounds awarding a Progress Prize and the final Netflix Prize awarded in 2009. The data provided was a collection of 2,817,131 triples consisting of (`user`, `movie`, `date Rated`). And half of these additionally included the rating itself. Notice that as in our preceding example, the user-item information is nearly enough to specify the problem. In this particular dataset, the date

was provided. Later, we will dig into how time might be a factor, and in particular, for sequential recommendation systems.

The stakes were quite high in this competition. Requirements for beating the internal performance were a 10% increase in root mean square error (RMSE); we will discuss this loss function later. And the spoils added up to over \$1.1 million. The final winners were BellKor's Pragmatic Chaos (which incidentally won the two previous Progress Prizes) with a test RMSE of 0.8567. In the end, only a 20-minute earlier submission time kept BellKor ahead of the competitors The Ensemble.

To read in detail about the winning submissions, check out [“The BigChaos Solution to the Netflix Grand Prize”](#) by Andreas Töschel and Michael Jahrer and [“The BigChaos Solution to the Netflix Prize 2008”](#) by the same authors. Meanwhile, let's review a few important lessons from this competition:

First, we see that the user-item matrix we've discussed appears in these solutions as the critical mathematical data structure. The model selection and training is important, but parameter tuning provided a huge improvement in several algorithms. We will return to parameter tuning in later chapters. The authors state that several model innovations came from reflecting on the business use case and human behavior and trying to capture those patterns in the model architectures. Next, linear-algebraic approaches resulted in the first reasonably performant solutions, and building on top of them led to the winning model. Finally, eking out the performance that Netflix originally demanded to win the competition took so long that business circumstances changed and the [solution was no longer useful](#).

That last point might be the *most* important thing an ML developer needs to learn about recommendation systems; see the following tip.

START WITH SIMPLICITY

Build a working usable model quickly and iterate while the model is still relevant to the needs of the business.

Soft Ratings

In our cheese-tasting example, each cheese either received a numerical rating or was not tried by a guest. These are *hard ratings*: regardless of whether the cheese is a brie or a chèvre, the ratings are explicit, and their absence indicates a lack of interaction between the user and item. In some contexts, we'll want to accommodate data indicating a user does interact with an item and yet provides no rating.

A common example is a movies app; a user may have watched a movie with the app but not provided a star rating. This indicates that the item (in this case, a movie) has been observed, but we don't have the rating for our algorithms to learn from. However, we can still use this implicit data to do the following:

- Exclude this item from future recommendations
- Use this data as a separate term in our learner
- Assign a default rating value to indicate “interesting enough to watch, not significant enough to rate”

It turns out that implicit ratings are critical for training effective recommendation systems, not only because users often don't give hard ratings, but also because implicit ratings provide a different level of signal. Later, when we wish to train multilevel models to predict both click likelihood and buy likelihood, these two levels will prove extremely important.

To sum up:

- A hard rating occurs when the user directly responds to a prompt for feedback on an item.
- A soft rating occurs when the user's behavior implicitly communicates feedback on an item without responding to a direct prompt.

Data Collection and User Logging

We've established that we learn from both explicit ratings and implicit ratings, so how and where do we get this data? To dive into this, we'll need to start worrying about application code. In many businesses, the data scientists and ML engineers are separate from the software engineers, but working with recommendation systems requires alignment between the two functions.

What to Log

The simplest and most obvious data collection is user ratings. If users are given the option to provide ratings, or even a thumbs-up or thumbs-down, that component will need to be built and that data will need to be stored. These ratings must be stored not only for the opportunity to build recommendations, but also to prevent the bad user experience of rating something and then shortly thereafter not having the rating appear when revisiting the page.

Similarly, it's useful to understand a few other key interactions that can improve and expand your recommendation system: page loads, page views, clicks, and add-to-bag.

For these types of data, let's use a slightly more complicated example: the ecommerce website Bookshop.org. This one site has multiple applications of recommendation systems, almost all of which we will return to in time. For now, let's focus on some interactions ([Figure 2-2](#)).

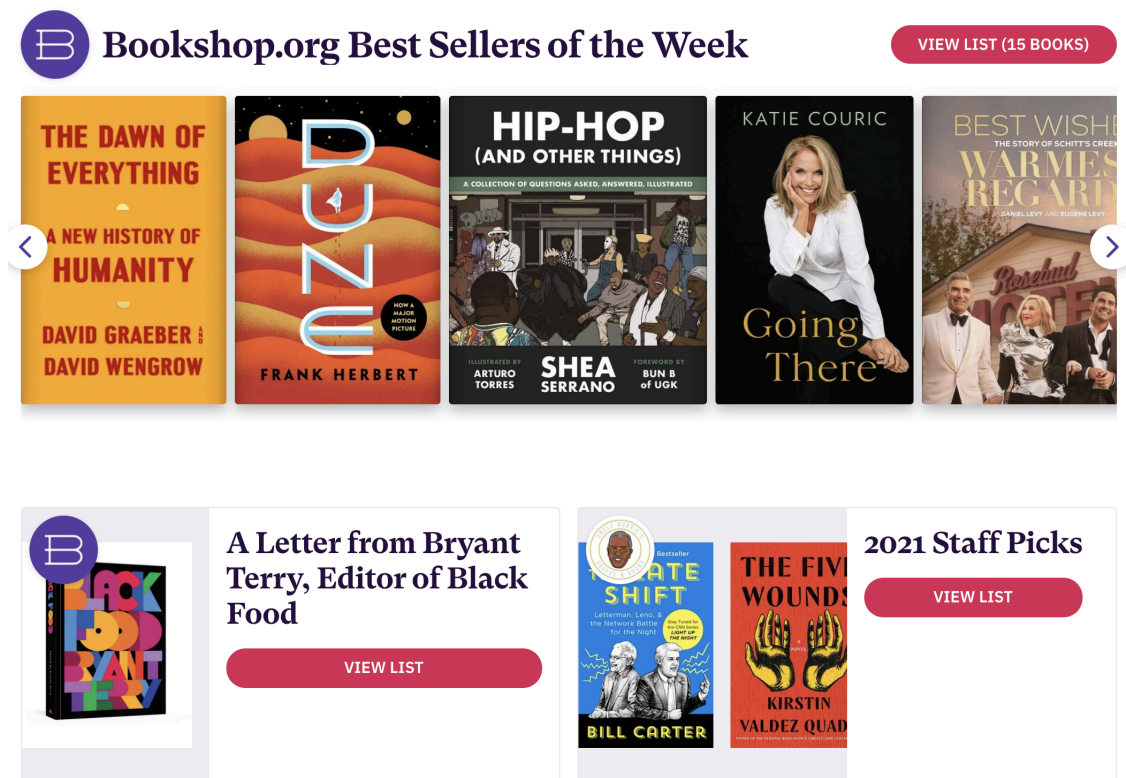


Figure 2-2. Bookshop.org landing page

Page loads

When you first load up Bookshop.org, it starts with items on the page. The Best Sellers of the Week are all clickable images to those book listings. Despite the user having no choice in loading this initial page, it's actually quite important to log the contents of this initial page load.

These options represent the population of books that the user has seen. If a user has seen an option, they have the opportunity to click it, which will ultimately be an important implicit signal.

PROPENSITY SCORES

The consideration of the population of all items a user has seen is deeply tied to propensity score matching. In mathematics, *propensity scores* are the probability that an observational unit will be assigned to the treatment group versus the control group.

Compare this setup to the simple 50-50 A/B test: every unit has a 50% chance of being exposed to your treatment. In a feature-stratified A/B test, you purposely change the probability of exposure dependent on a certain feature or collection of features (often called *covariates* in this context). Those probabilities of exposure are the propensity scores.

Why bring up A/B testing here? Later, we'll be interested in mining our soft ratings for signal on user preference, but we must consider the possibility that the lack of a soft rating is not an implicit bad rating. Thinking back to the cheeses: taster *D* never had a chance to rate *brie*, so there's no reason to think *D* has a preference for aversion on *brie*. This is because *D* was not *exposed to brie*.

Now thinking back to Bookshop.org: the landing page does not show *The Hitchhiker's Guide to the Galaxy*, so the user has no way to click it and implicitly communicate interest in that book. The user could use the search option, but that's a different kind of signal—which we'll talk about later and is, in fact, a much stronger signal.

When understanding implicit ratings like “did the user look at something,” we need to properly account for the entire population of choices they were exposed to, and use the inverse of that population size to weigh the importance of clicking. For this reason, understanding *all* page loads is important.

Page views and hover

Websites have gotten much more complicated, and now users must contend with a variety of interactions. [Figure 2-3](#) demonstrates what happens if the user clicks the right arrow in the Best Sellers of the Week carousel and then moves their mouse over the Cooking at Home option.



Figure 2-3. Bookshop.org top sellers

The user has unveiled a new option, and by mousing over it, has made it larger and given it a visual effect. These are ways to communicate more information to the user, and remind the user that these options are clickable. To the recommender, these clicks can be used as more implicit feedback.

First, the user clicked the carousel scroll—so some of what they saw in the carousel was interesting enough to dig further. Second, they moused over *Cooking at Home*, which they might click or might just want to see if additional information becomes available when hovering. Many websites use a hover interaction to provide a pop-up detail. While Bookshop.org doesn't implement something like this, internet users have been trained to expect this behavior by all the websites that do, and so the signal is still meaningful. Third, the user has now uncovered a new potential item in their carousel scroll—which we should add to our page loads but with a higher rating because it required interaction to uncover.

All this and more can be encoded into the website's logging. Rich and verbose logging is one of the most important ways to improve a recommendation system. Having more logging data than you need is almost always better than having the opposite.

Clicks

If you thought hovering meant interest, wait until you consider clicking! Not in all cases, but in the large majority, clicking is a strong indicator of product interest. For ecommerce, clicking often is computed as part of the recommendation team's core key performance indicators (KPIs).

This is for two reasons:

- Clicking is almost always required to purchase, so it's an upstream filter for most business transactions.
- Clicking requires explicit user action, so it's a good measure of intent.

Noise will always exist of course, but clicks are the go-to indicator of a client's interest. Many production recommendation systems are trained on click data—not ratings data—because of the much higher data volume and the strong correlation between click behavior and purchase behavior.

CLICK-STREAM DATA

Sometimes in recommendation systems you hear people talk about *click-stream* data. This important view into click data also considers the order of a user's clicks in a single *session*. Modern recommendation systems put a lot of effort into utilizing the order of items a user clicks, calling this *sequential recommendations*, and have shown dramatic improvements via this additional dimension. We will discuss sequence-based recommendations in [Chapter 7](#).

Add-to-bag

We've finally arrived; the user has added an item to their bag or cart or queue. This is an extremely strong indicator of interest and is often quite correlated with purchasing. There are even reasons to argue that add-to-bag is a better signal than purchase/order/watch. Add-to-bag is essentially the end of the line for soft ratings, and usually beyond this you'd want to start collecting ratings and reviews.

Impressions

We might also wish to log *impressions* of an item that wasn't clicked. This supplies the recommendation system with negative feedback on items that the user isn't interested in. For example, if the cheeses *gouda*, *chèvre* and *emmentaler* are offered to the user but the user tastes only *chèvre*, perhaps the user doesn't like *gouda*. They might not have gotten around

to tasting *emmentaler*, on the other hand, so these impressions may carry only noisy signal.

Collection and Instrumentation

Web applications frequently instrument all the interactions we’ve discussed via events. If you don’t yet know what events are, maybe ask a buddy in your engineering org—but we’ll also give you the skinny. Like logging, *events* are specially formatted messages that the application sends out when a certain block of code is executed.

As in the example of a click, the application needs to make a call to get the next content to show the user, it’s common to also “fire an event” at this moment, indicating information about the user, what they clicked, the session-ID for later reference, the time, and various other useful details. This event can be handled downstream in any number of ways, but there’s an increasingly prevalent pattern of path bifurcation to the following:

- A log database, like a mySQL application database tied to the service
- An event stream for real-time handling

The latter will be interesting: event streams are often connected to listeners via technologies like Apache Kafka. This kind of infrastructure can get complicated fast (consult your local data engineer or MLOps person), but a simple model for what happens is that all of a particular kind of log are sent to several destinations that you think can make use of these events.

In the recommender case, an event stream can be connected up to a sequence of transformations to process the data for downstream learning tasks. This will be enormously useful if you want to build a recommendation system that uses those logs. Other important uses are real-time metrics logging for what is going on at any given time on the website.

Funnels

We've just worked through our first example of a funnel, which no good data scientist can avoid thinking about. Like them or hate them, funnel analyses are crucial for critical evaluation of your website, and by extension your recommendation system.

CLICK-STREAMS

A *funnel* is a collection of steps a user must take to get from one state to another; it's called a funnel because at each of the discrete steps, a user may stop proceeding through, or *drop off*, thus reducing the population size at each step.

In our discussion of events and user logging, each step is relevant for a subset of the previous. This means that the process is a funnel, as shown in [Figure 2-4](#). Understanding the drop-off rate at each step reveals important characteristics of your website and your recommendations.

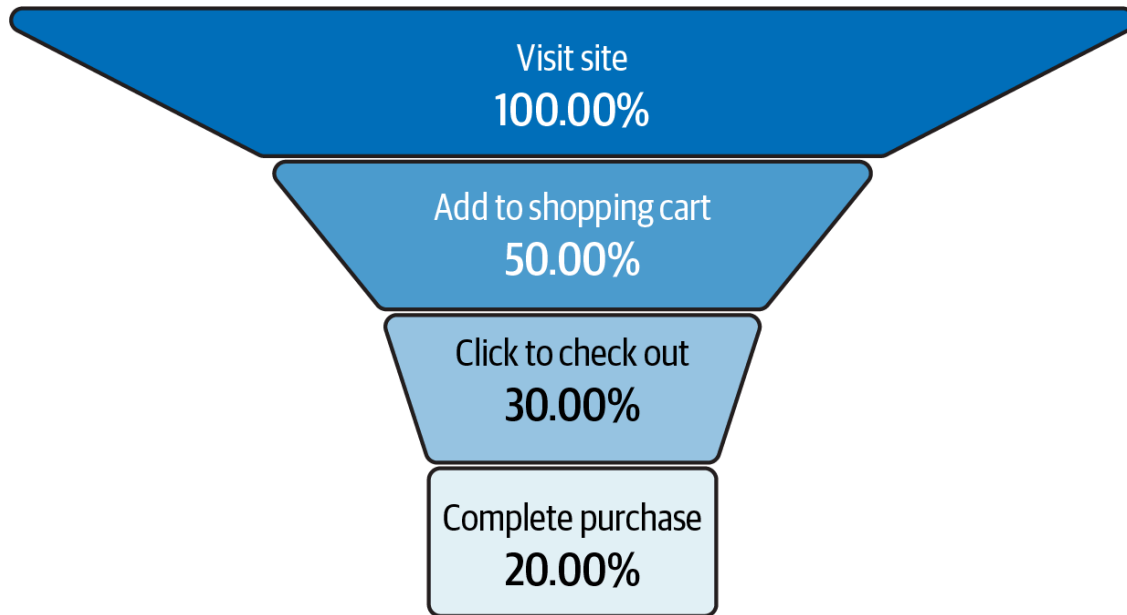


Figure 2-4. An onboarding funnel

Three important funnel analyses can be considered in [Figure 2-4](#):

1. Page view to add-to-bag user flow
2. Page view to add-to-bag per recommendation
3. Add-to-bag to complete purchase

The first funnel is merely identifying, at a high level, the percentage of users who take each step in the flow. This is a high-level measure of your website optimization, the general interestingness of your product offering, and the quality of your user leads.

The second funnel, which is more fine-grained, takes into consideration the recommendations themselves. As mentioned previously in terms of propensity scoring, users can proceed through the funnel for a particular item only if they're shown the item. This concept intersects with the use of funnels because you want to understand at a high level how certain recommendations correlate with funnel drop-off, but also, when using a recommendation system, the confidence in your recommendations should correlate well with the funnel metrics. We will return to this in more detail in [Part III](#), but for now you should remember to think about different categories of recommendation-user pairs and how their funnels may look compared to the average.

Finally, we can consider add-to-bag to completion. This actually isn't part of the RecSys problem but should be on your mind as a data scientist or ML engineer trying to improve the product. *No matter how good your recommendations are, this funnel may destroy any of your hard work.* Before working on a recommender problem, you should almost always investigate the funnel performance in getting a user from add-to-bag to check-out-completed. If there's something cumbersome or difficult about this flow, it will almost certainly provide a bigger bang for your buck to fix this than to improve recommendations. Investigate the drop-offs, do user studies to understand what might be confusing, and work with product and engineering teams to ensure that everyone is aligned on this flow before you start building a recommender for ecommerce.

Business Insight and What People Like

In the previous example from Bookshop.org, Top Sellers of the Week is the primary carousel on the page. Recall our earlier work on `get_most_popular_recs` ; what powers the carousel is simply that recom-

mender but applied to a specific collector—one that looks only in the last week.

This carousel is an example of a recommender providing business insight in addition to driving recommendations. A common mission of a growth team is to understand weekly trends and KPIs, often metrics like weekly active users and new sign-ups. For many digital-first companies, growth teams are additionally interested in understanding the primary drivers of engagement.

Let's take an example: as of this writing, the Netflix show *Squid Game* became the company's most popular series of all time, breaking a huge number of records in the process. *Squid Game* reached 111 million viewers in the first month. Most obviously, *Squid Game* needs to be featured in the Top Shows of the Week or Hottest Titles carousels, but where else should a breakout hit like this matter?

The first important insight companies almost always ask for is *attribution*: if the numbers go up in a week, what led to that? Is there something important or special about launches that drove additional growth? How can we learn from those signals to do better in the future? In the case of *Squid Game*—a foreign-language show that saw massive interest from an English-speaking audience—executives might take away the inclination to invest more in shows from South Korea or in subtitled shows with high drama. The flip side of this coin is also important: when growth metrics lag, executives nearly always ask why. Being able to point to what was the most popular, and how it may have deviated from expectation, helps a lot.

The other important insight can feed back into recommendations; during exciting debuts like *Squid Game*, it's easy to get caught up in the excitement as you see all your metrics go up and to the right, but might this negatively affect metrics also? If you have a show debuting the same week or two as *Squid Game*, you'll be less enthusiastic about all this success. Overall, successes like this usually drive *incremental* growth, which is great for business, and in total, metrics will all probably look up. Other items, however, may have less successful launches due to a zero-sum

game among the core user base. This can have a negative effect on longer-term metrics and can even make later recommendations less effective.

Later, you will learn about diversity of recommendations; there are many reasons to care about diversifying your recommendations, but here we observe one: diversifying can increase the overall ability to match your users with items. As you keep a broad base of users highly engaged, you increase your future opportunity for growth.

INCREMENTAL GAINS

Incremental gains is an economics term now used in growth marketing and growth analytics. Incremental gains refer to a margin of increase in addition to the gains expected from an expended effort.

A simple example is a business that usually adds a user for every \$100 in marketing spending, gets some positive press, and the next week gets a user for every \$80 in marketing spending. By keeping the marketing budget fixed that week at \$1,600, the business would get 20 new users instead of 16—an incremental gain of 4 users. This framework is especially common when testing new treatments or programs.

Finally, beyond surfacing the trending hits, another benefit of knowing what's really hot on your platform or service is advertising. When a phenomenon starts, a huge advantage can result from priming the pump—making noise and driving publicity of the success. This sometimes leads to a network effect, and in these days of viral content and easy distribution, this can have multiplicative impacts on your platform's growth.

Summary

This constitutes the most basic aspects of formulating your recommendation problems and preparing yourself to solve them.

The user-item matrix gave us a tool to summarize the relationship between users and items in the simplest case of numerical ratings and will generalize to more complicated models later. We saw our first notion of vector similarity, which will be expanded to a deep geometric notion of relevance. Next, we learned about the kinds of signals that users can provide via explicit and implicit actions. Finally, we learned how to capture these actions for training models.

Now that we've finished our problem framing, we've got a bit of a math review for you. Don't worry, you can keep your ruler and compass packed away, and you won't be required to prove anything or compute any integrals. You will, however, see some important mathematical notions that will help you think clearly about expectations for your recommendation systems and ensure you're asking the right questions.